SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING, KALAVAKKAM
(An Autonomous Institution, Affiliated to Anna University, Chennai)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
III Year CSE (V Semester)
Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Lab Exercise 7 - Design and Simulate an Autocomplete Search System using Prefix Matching

Objective

To implement an autocomplete search system using prefix matching in a Spring Boot application, use Redis for caching frequently searched prefixes, and analyze the performance of the system under different workloads.

Problem Statement

A search system needs to provide relevant suggestions as the user types a query. For example, when the user enters the prefix app, the system should provide suggestions such as apple, application, appointment, and app store.

As the number of search terms and users increases, searching through the complete collection for every request can increase the response time. To improve the efficiency of the system, implement prefix-based search using a Trie and use Redis to cache frequently requested prefixes and their suggestions.

Develop the autocomplete service using Spring Boot, expose the search functionality through a REST API, and evaluate the system performance using JMeter.

Tasks

- Create a dataset containing search terms and their corresponding search frequencies.
- Develop a Spring Boot application implementing the autocomplete search functionality.
- Implement a Trie to store and retrieve search terms based on their prefixes.
- Retrieve the matching search suggestions for a given prefix.
- Return the top-K suggestions based on search frequency.
- Integrate Redis to cache frequently requested prefixes and their suggestions.
- Retrieve suggestions from Redis when the requested prefix is available in the cache.
- Perform Trie-based search when the requested prefix is not available in Redis.
- Store the generated suggestions in Redis for subsequent requests.

- Test the autocomplete REST API using Postman.
- Deploy the application and Redis using Docker.
- Use JMeter to generate concurrent autocomplete requests.

Analysis

1. Explain the role of:
    - Trie
    - Prefix
    - Redis Cache
    - Top-K Suggestions
2. Explain how the Trie determines the search suggestions for a given prefix.
3. Explain how the system processes an autocomplete request when the prefix is available in Redis.
4. Explain how the system processes an autocomplete request when the prefix is not available in Redis.
5. Compare Trie-based prefix searching with linear searching.

Expected Learning Outcomes

After completing this experiment, students will be able to:

- Develop an autocomplete search system using Spring Boot.
- Implement prefix-based searching using a Trie.
- Implement Top-K search suggestions.
- Integrate Redis caching into an application.
- Analyze the performance improvement achieved through caching.